

ConcertHub

Object-Oriented Design & Coded Solution — HD Working Plan

A merged scenario, use-case set, class design and starter skeleton built from both team members' individual written submissions, structured against the official assessment rubric.

Team member	Individual topic write-up used
Jordan Wong	Budget concert booking platform for upcoming artists
Daniel Lukins	Concert technical/seating/ticketing management

This document is a **working draft** for your group, not a finished submission. The brief requires the two of you to jointly agree the final scenario and use-cases (min. 4 for a group of 2) — treat everything here as the starting proposal to discuss, edit and sign off together, then paste the agreed version into your official submission document alongside the required cover sheet.

1. How This Plan Maps to Your Marking Rubric

Project A is graded on six criteria. Here's what in this document targets each one, and what still needs **your own explanation in your own words** — a marker can tell when explanations are copy-pasted rather than understood, and criterion 4 explicitly grades whether you can justify your own choices.

Criteria	Weight	Covered by
Class relationships	15%	§6 Class Design + diagram — shows every class, inheritance line, and interface realisation.
Functional specifications	25%	§5 Functional Requirement Map + the working skeleton in <code>src/</code> — you must finish the TODOs so the program actually runs correctly.
Design rules	25%	Skeleton code follows camelCase/PascalCase, toString() on every class, sparing setters, and only week 1–6 constructs (see §9 warning).
Scenario & explanation, justifying choices	5%	§3 Scenario, §4 Use-Cases, §7 OOP Concept Crib Notes — you write the final explanation prose yourselves using these as a scaffold.
Write cohesively (in-class)	15%	Individual, in-class, no prep possible — see §10.
Content-specific lexicogrammar (in-class)	15%	Individual, in-class — see §10.

Note on the two in-class criteria (30% combined): these are assessed under exam conditions with pen and paper, individually, on your *own* scenario paragraph and use-cases — no electronic devices allowed. No amount of preparation here replaces practising writing a tight 4–5 sentence business scenario and clear use-case sentences by hand, quickly. §10 has a short drill for this.

2. Original Individual Submissions

Per the brief, the group scenario must be built by "taking stock of each member's submission." Here's what you each wrote in week 5, for reference when you write the explanation section later.

Jordan Wong — Topic: Concert

"The business for this case would be a function spot which gives smaller artists an opportunity to host events to their fans. It is solely for a more 'budget' version for upcoming artists who might not have the funds right now to host something at a big/popular location like Qudos Arena. This business will have a very thorough and proper online system/program which will let artists customise costs, equipment, numbers, etc."

Use-cases: book available dates · select equipment · fans track upcoming shows · view budget/profit · buy ticket or register interest.

Daniel Lukins — Topic: Concert

"Taylor Swift is holding a concert featuring various other artists. To manage this concert, a program will be created for the concert technical team. It will allow the concert team to view the order of songs to be played, which tickets are available, when certain sound effects to be played, etc... a large scale of tickets need to be managed."

Use-cases: seat availability display · seating updates as tickets sell · manage song order · view ticket revenue · manage ticket type details.

3. Proposed Merged Scenario

ConcertHub is an online platform that helps independent and emerging artists book budget-friendly performance slots at partner venues, without the overheads of a large arena booking. When an artist reserves a slot, ConcertHub lets them select the sound and stage equipment they require and tracks that equipment cost against the booking. Fans can browse upcoming shows, purchase or reserve a numbered seat, and see which of their favourite artists have concerts coming up. Venue staff use the same system to open ticket sales for a concert, monitor real-time seat availability, and manage the running order of songs for the night, giving artists a live view of their projected profit as tickets sell.

This directly combines Jordan's booking/equipment/budget angle with Daniel's seating/ticketing/song-order angle into one coherent business, rather than picking one and discarding the other. Discuss and edit this together — you both need to be able to explain *why* the final wording was chosen, referencing your original paragraphs.

4. Proposed Use-Cases (7)

Groups of 2 need a **minimum of 4**. Seven is proposed here to comfortably cover the HD band for "functional specifications" and give you room to cut one back if time is tight in week 7. Remember the brief's warning: add/remove/modify of the *same* thing counts as one use-case, not several — each item below is written as a single sentence for that reason.

#	Use-case	Traces back to
1	Artists can create a new concert booking by selecting an available date and venue slot.	Jordan UC1
2	Artists can add or remove the equipment items they need for a booking (e.g. drum kit, piano, speakers, microphone), with the running equipment cost shown.	Jordan UC2
3	Venue staff can open a seating chart for a concert and see, at a glance, which seats are available or sold.	Daniel UC1, UC2
4	Fans can purchase or reserve a ticket for a specific concert and seat, updating seat availability immediately.	Jordan UC5, Daniel UC2
5	Artists and venue staff can view a budget summary for a booking (equipment cost vs. ticket revenue collected so far).	Jordan UC4, Daniel UC4
6	Venue staff can manage the running order of songs for a concert.	Daniel UC3
7	Fans can browse upcoming concerts sorted by date, and filter the list down to a single artist's shows.	Jordan UC3, Daniel scenario

If your tutor gives you a different topic in week 5 (the brief says a topic will be assigned, e.g. "a café, or a library" — concert may or may not be the actual assigned topic), the class design in §6 still transfers with light renaming: `Booking→Order` , `Artist→Customer` , `Equipment→Item` , `Ticket→LineItem` . The underlying pattern (a hub object owning a list, a map, and two cost-bearing item types under one interface) works for most "manage bookings/orders + inventory + people" topics.

5. Functional Requirement Map

The brief requires the program to respond to keyboard input for four specific operations, at minimum. Here's where each one lives in the skeleton:

Brief requirement	Where it's implemented
Add / remove objects to collections	<code>Booking.addEquipment()</code> / <code>removeEquipment()</code> , <code>Booking.addSong()</code> / <code>removeSong()</code> , <code>Main.createBooking()</code> adding to the <code>bookings</code> list
Retrieve a specific object from a collection	<code>Main.findBookingByDate(String date)</code> — linear search over <code>bookings</code>
Sort a collection and display its objects	<code>Main.browseConcerts()</code> — <code>Collections.sort(bookings)</code> , driven by <code>Booking</code> implements <code>Comparable<Booking></code>
Create a sub-collection from a larger collection	<code>Booking.getAvailableSeats()</code> — filters the unsold entries out of <code>seatMap</code> into a new <code>ArrayList<Ticket></code>

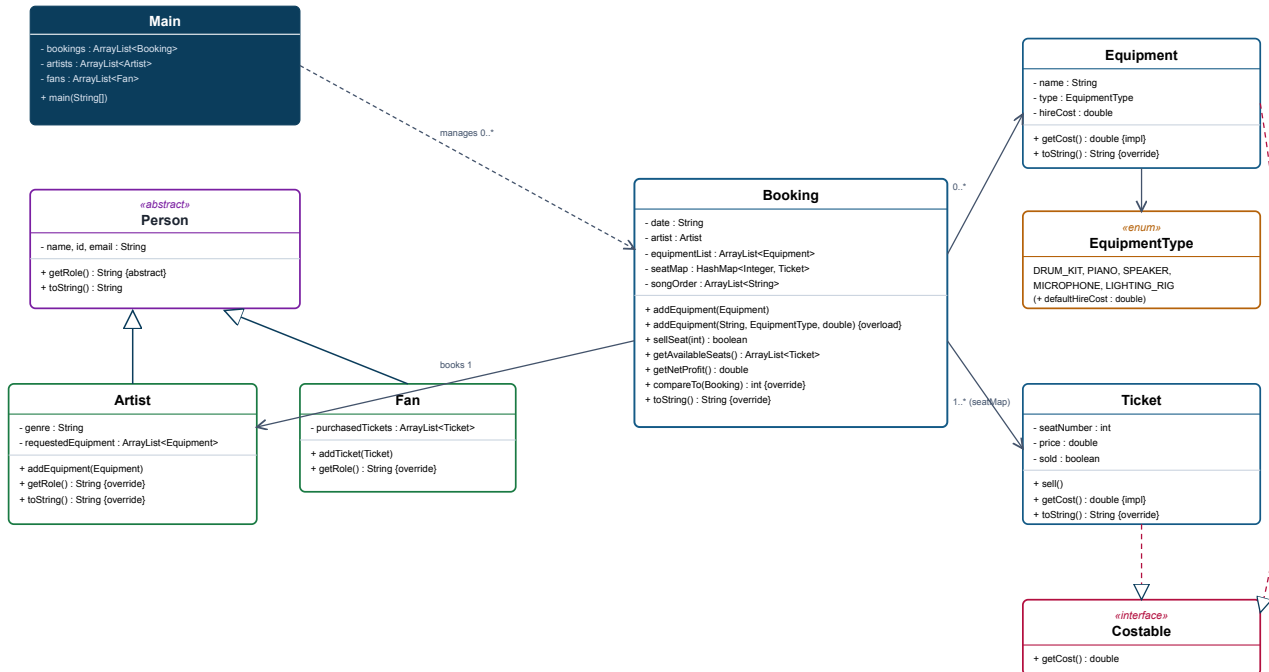
Plus the subject objective to demonstrate **both** list and map structures: `ArrayList` is used throughout (bookings, equipment, song order); `HashMap<Integer, Ticket>` is used for `Booking.seatMap` , keyed by seat number — this is also a direct upgrade of the 2D-array seating grid already sketched in the earlier `Main.java` / `SeatingView` commit, now expressed as the map structure the objectives ask for.

6. Class Design

Nine files, sized for a 2-person group (brief target: 6–8 classes, including the driver, ≥ 1 parent, ≥ 2 children, ≥ 1 enum). `Costable` is an interface rather than a class and typically isn't counted toward that range.

File	Role	Extends / Implements
Main.java driver	Text-menu entry point; owns the master <code>ArrayList s</code>	—
Person.java abstract parent	Shared identity fields for anyone in the system	—
Artist.java child	A performer who books a slot and requests equipment	extends Person
Fan.java child	An attendee who buys tickets	extends Person
Booking.java	The hub: one concert event, its equipment, its seats, its song order	implements <code>Comparable<Booking></code>
Equipment.java	A hireable stage/sound item	implements <code>Costable</code>
Ticket.java	One seat within a booking	implements <code>Costable</code>
EquipmentType.java enum	Fixed set of equipment kinds, each with a default hire cost	—
Costable.java interface	Anything with a dollar cost that can be summed for the budget view	—
In.java	Scanner input helper (already drafted by the team)	—

Simplified class diagram



Solid hollow-triangle arrows = inheritance (`extends`). Dashed hollow-triangle arrows = interface realisation (`implements`). Plain arrows = association/uses. This is intentionally "simplified" per the brief — it omits helper getters and only shows the methods worth discussing in your explanation.

7. OOP Concept Crib Notes (for "The Explanation")

The brief requires you to identify and justify six things in your write-up. Use these as a starting scaffold — **rewrite them in your own words** once the code is actually finished; a marker can tell generic justifications from ones that reference your actual working code.

Inheritance

`Artist` and `Fan` both extend the abstract class `Person`. Both need a name, an ID and an email, and both need a role label — pulling those into one parent avoids duplicating the same three fields and `toString()` logic twice. Because a fan and an artist are genuinely different kinds of person with different behaviour (an artist requests equipment, a fan buys tickets), inheritance is a better fit here than one flat class with unused fields.

Interfaces

`Costable` is implemented by both `Equipment` and `Ticket` — two classes that have nothing else in common (one is stage gear, one is a seat) but both contribute a dollar figure to a booking's budget. An interface lets `Booking` treat them polymorphically wherever only the cost matters, without forcing them into a shared inheritance hierarchy they don't otherwise belong in. `Booking` also implements the built-in `Comparable<Booking>` interface so a list of bookings has a natural sort order (by date) for `Collections.sort()`.

Overloaded methods

`Booking.addEquipment(Equipment)` is overloaded with `addEquipment(String name, EquipmentType type, double cost)` — the second form saves the caller from having to construct an `Equipment` object first when they already have the raw details from user input. Same idea for `Artist`'s two constructors: one lets a caller skip specifying a genre and defaults it to `"Unspecified"`.

Overriding methods

Every class overrides `toString()` so each object prints something meaningful instead of Java's default `Object@hashCode`. `Person.getRole()` is `abstract` and each subclass overrides it differently ("Artist" vs "Fan") — this is what lets `toString()`, defined once in `Person`, produce a different label per subclass automatically. `Booking.compareTo()` overrides the interface method to define "earlier date first" ordering.

Enum classes

`EquipmentType` is an enum rather than a set of `String` constants because the set of valid equipment kinds is fixed and known in advance — using an enum makes invalid values (like a typo'd equipment name) impossible to construct, and lets each constant carry its own `defaultHireCost`, which a plain `String` couldn't do.

Polymorphism

Anywhere the code holds a `Person` reference (e.g. iterating a mixed list of people), calling `.getRole()` or `.toString()` resolves to the actual runtime subclass's version — an `Artist` in a `Person` variable

still prints "Artist ...", not something generic. The same applies to `Costable` : if you build a single `ArrayList<Costable>` containing both `Equipment` and `Ticket` objects (a nice stretch addition to `getTotalEquipmentCost()` / `getTotalRevenue()`), calling `.getCost()` on each runs the correct class's implementation without the calling code needing to know which one it is.

8. Skeleton Code

These files already exist in your project's `src/` folder and compile as-is (`javac *.java` — verified). Method bodies marked `// TODO` are left for you and your partner to implement together during your ~10 hours of group work — that's the part the brief actually asks you to build and be able to explain, so it's intentionally not pre-solved here.

Costable.java

```
public interface Costable {  
    double getCost();  
}
```

EquipmentType.java

```
public enum EquipmentType {  
    DRUM_KIT(150.00),  
    PIANO(300.00),  
    SPEAKER(80.00),  
    MICROPHONE(25.00),  
    LIGHTING_RIG(200.00);  
  
    private final double defaultHireCost;  
  
    EquipmentType(double defaultHireCost) {  
        this.defaultHireCost = defaultHireCost;  
    }  
  
    public double getDefaultHireCost() {  
        return defaultHireCost;  
    }  
}
```

Person.java

```
public abstract class Person {
    private String name;
    private String id;
    private String email;

    public Person(String name, String id, String email) {
        this.name = name;
        this.id = id;
        this.email = email;
    }

    public String getName() { return name; }
    public String getId() { return id; }
    public String getEmail() { return email; }

    public abstract String getRole();

    @Override
    public String toString() {
        return getRole() + " " + name + " (ID: " + id + ", " + email + ")";
    }
}
```

Artist.java

```

import java.util.ArrayList;

public class Artist extends Person {
    private String genre;
    private ArrayList<Equipment> requestedEquipment;

    public Artist(String name, String id, String email) {
        this(name, id, email, "Unspecified");
    }

    public Artist(String name, String id, String email, String genre) {
        super(name, id, email);
        this.genre = genre;
        this.requestedEquipment = new ArrayList<>();
    }

    public String getGenre() { return genre; }
    public ArrayList<Equipment> getRequestedEquipment() { return requestedEquipment; }

    public void addEquipment(Equipment item) {
        // TODO: add item to requestedEquipment
    }

    public boolean removeEquipment(Equipment item) {
        // TODO: remove item from requestedEquipment, return whether it was removed
        return false;
    }

    @Override
    public String getRole() { return "Artist"; }

    @Override
    public String toString() { return super.toString() + " | Genre: " + genre; }
}

```

Fan.java

```

import java.util.ArrayList;

public class Fan extends Person {
    private ArrayList<Ticket> purchasedTickets;

    public Fan(String name, String id, String email) {
        super(name, id, email);
        this.purchasedTickets = new ArrayList<>();
    }

    public ArrayList<Ticket> getPurchasedTickets() { return purchasedTickets; }

    public void addTicket(Ticket ticket) {
        // TODO: add ticket to purchasedTickets
    }

    @Override
    public String getRole() { return "Fan"; }
}

```

Equipment.java

```

public class Equipment implements Costable {
    private String name;
    private EquipmentType type;
    private double hireCost;

    public Equipment(EquipmentType type) {
        this(type.toString(), type, type.getDefaultHireCost());
    }

    public Equipment(String name, EquipmentType type, double hireCost) {
        this.name = name;
        this.type = type;
        this.hireCost = hireCost;
    }

    public String getName() { return name; }
    public EquipmentType getType() { return type; }

    @Override
    public double getCost() { return hireCost; }

    @Override
    public String toString() { return name + " (" + type + ") - $" + hireCost; }
}

```

Ticket.java

```
public class Ticket implements Costable {
    private int seatNumber;
    private double price;
    private boolean sold;

    public Ticket(int seatNumber, double price) {
        this.seatNumber = seatNumber;
        this.price = price;
        this.sold = false;
    }

    public int getSeatNumber() { return seatNumber; }
    public boolean isSold() { return sold; }

    public void sell() {
        // TODO: mark this ticket as sold
    }

    @Override
    public double getCost() { return price; }

    @Override
    public String toString() {
        return "Seat " + seatNumber + " - $" + price + (sold ? " [SOLD]" : " [AVAILABLE]");
    }
}
```

Booking.java

```

import java.util.ArrayList;
import java.util.HashMap;

public class Booking implements Comparable<Booking> {
    private String date;
    private Artist artist;
    private ArrayList<Equipment> equipmentList;
    private HashMap<Integer, Ticket> seatMap;
    private ArrayList<String> songOrder;

    public Booking(String date, Artist artist, int totalSeats) {
        this.date = date;
        this.artist = artist;
        this.equipmentList = new ArrayList<>();
        this.seatMap = new HashMap<>();
        this.songOrder = new ArrayList<>();
        // TODO: populate seatMap with `totalSeats` Ticket objects (seat numbers 1..totalSeats)
        //         decide on a base ticket price, e.g. new Ticket(i, 45.00)
    }

    public String getDate() { return date; }
    public Artist getArtist() { return artist; }

    public void addEquipment(Equipment item) {
        // TODO
    }

    public void addEquipment(String name, EquipmentType type, double cost) {
        // TODO: build a new Equipment from the given details and add it (overload of addEquipment)
    }

    public boolean removeEquipment(Equipment item) {
        // TODO
        return false;
    }

    public boolean sellSeat(int seatNumber) {
        // TODO: look up the ticket in seatMap, mark it sold if it exists and isn't already sold
        //         return true on success, false otherwise
        return false;
    }

    public ArrayList<Ticket> getAvailableSeats() {
        // TODO: build and return a new ArrayList containing only the unsold tickets from seatMap
        //         (this satisfies the "sub-collection" requirement in the brief)
        return new ArrayList<>();
    }

    public void addSong(String title) {
        // TODO
    }

    public void removeSong(String title) {
        // TODO
    }

    public void reorderSong(int fromIndex, int toIndex) {

```

```

        // TODO: move the song at fromIndex to toIndex within songOrder
    }

    public ArrayList<String> getSngOrder() { return songOrder; }

    public double getTotalEquipmentCost() {
        // TODO: sum getCost() across equipmentList
        return 0;
    }

    public double getTotalRevenue() {
        // TODO: sum getCost() across sold tickets in seatMap
        return 0;
    }

    public double getNetProfit() { return getTotalRevenue() - getTotalEquipmentCost(); }

    @Override
    public int compareTo(Booking other) { return this.date.compareTo(other.date); }

    @Override
    public String toString() {
        return date + " | " + artist.getName() + " | Net profit so far: $" + getNetProfit();
    }
}

```

Main.java

```

import java.util.ArrayList;
import java.util.Collections;

public class Main {
    private static ArrayList<Booking> bookings = new ArrayList<>();
    private static ArrayList<Artist> artists = new ArrayList<>();
    private static ArrayList<Fan> fans = new ArrayList<>();

    public static void main(String[] args) {
        int choice;
        do {
            printMenu();
            choice = In.nextInt();
            switch (choice) {
                case 1: createBooking(); break;
                case 2: manageEquipment(); break;
                case 3: sellTicket(); break;
                case 4: viewBudget(); break;
                case 5: manageSongOrder(); break;
                case 6: browseConcerts(); break;
                case 0: System.out.println("Goodbye!"); break;
                default: System.out.println("Invalid option, try again.");
            }
        } while (choice != 0);
    }

    private static void printMenu() {
        System.out.println("\n=== ConcertHub ===");
        System.out.println("1. Create a new booking");
        System.out.println("2. Manage equipment for a booking");
        System.out.println("3. Sell a ticket / seat");
        System.out.println("4. View budget summary for a booking");
        System.out.println("5. Manage song running order");
        System.out.println("6. Browse concerts (sorted by date)");
        System.out.println("0. Exit");
        System.out.print("Choose an option: ");
    }

    private static void createBooking() {
        // TODO: prompt for artist details (or pick an existing Artist from `artists`),
        //       prompt for date and number of seats, construct a Booking, add to `bookings`
    }

    private static void manageEquipment() {
        // TODO: ask which booking (retrieve a specific object from `bookings`),
        //       then add/remove Equipment on it
    }

    private static void sellTicket() {
        // TODO: ask which booking and seat number, call booking.sellSeat(seatNumber)
    }

    private static void viewBudget() {
        // TODO: ask which booking, print equipment cost, revenue and net profit
    }

    private static void manageSongOrder() {

```

```

        // TODO: ask which booking, then add/remove/reorder songs
    }

    private static void browseConcerts() {
        // TODO: Collections.sort(bookings) (uses Booking.compareTo), then print each one.
        //      Optionally filter to a single artist's bookings first (another sub-collection)
    }

    private static Booking findBookingByDate(String date) {
        // TODO: linear search `bookings` for a matching date, return null if not found
        //      (this satisfies the "retrieve a specific object" requirement in the brief)
        return null;
    }
}

```

In.java (already drafted – unchanged)

```

import java.util.*;

public class In {
    private static Scanner in = new Scanner(System.in);

    public static String nextLine() { return in.nextLine(); }
    public static char nextChar() { return in.nextLine().charAt(0); }
    public static char nextUpperChar() { return in.nextLine().toUpperCase().charAt(0); }

    public static int nextInt() {
        int i = in.nextInt();
        in.nextLine();
        return i;
    }

    public static double nextDouble() {
        double d = in.nextDouble();
        in.nextLine();
        return d;
    }
}

```

9. Stay Inside Weeks 1–6

The brief is explicit and highlighted in yellow: "you must only use code constructs taught in weeks 1 to 6 inclusive. Using constructs outside of week 1 to 6 will substantially lower the marks you receive." This skeleton deliberately avoids streams, lambdas, generics wildcards, and try/catch, sticking to: classes, objects, constructors, primitives, `String`, arrays, `if / else`, loops, `ArrayList`, `HashMap`, inheritance, abstract classes, interfaces, enums, method overloading/overriding, and `Scanner`-based input.

Check this against your actual week-by-week syllabus before you submit — if `HashMap`, `enum`, or interfaces haven't actually been covered by week 6 in your tutorial, adjust: a `HashMap<Integer, Ticket>` can be swapped for a plain array of seats (closer to the original `SeatingView` sketch) if maps aren't examinable yet, and lists/loops alone will still satisfy the "Functional specifications" HD band.

10. Suggested 10-Hour Work Split (2 People)

Hours	Activity	Suggested owner
0.5	Finalise scenario + use-cases together (from your week 5 in-class pieces)	Both
1.0	Walk the class diagram together, agree any renames, confirm week 1–6 constructs	Both
1.5	Implement <code>Person</code> , <code>Artist</code> , <code>Fan</code> , <code>EquipmentType</code> , <code>Costable</code> , <code>Equipment</code>	Person A
1.5	Implement <code>Ticket</code> , <code>Booking</code> (seatMap, budget methods, song order)	Person B
1.5	Implement <code>Main</code> 's menu handlers, wiring both halves together	Both (pair)
1.5	Manual test pass — run every menu option, capture VS Code screenshots, fix bugs found	Both
1.5	Write the explanation document using §7 as a scaffold, in your own words	Both (split sections)
0.5	Fill in cover sheet, contribution list, final proofread	Both

Contribution list template (required by the brief)

Student	Contribution
Jordan Wong	— Wrote classes: ____ — Wrote sections: ____ of the explanation — Testing: ____
Daniel Lukins	— Wrote classes: ____ — Wrote sections: ____ of the explanation — Testing: ____

In-class writing drill (individual, no notes allowed on the day)

Before week 5, each of you should practise — by hand, no laptop — writing a 4–5 sentence business scenario and 5 single-sentence use-cases for a *random* topic (café, gym, library, pet shelter) in under 15 minutes. The goal isn't memorising content (you'll get an unknown topic on the day) but building the muscle of: state the business in sentence 1, state who uses it and why in sentences 2–3, state the gap it fills in sentence 4–5, then write use-cases as single, non-trivial sentences.

11. Testing & Final Submission Checklist

- ☐ Every menu option in `Main` runs without crashing, for both a valid and an invalid input
- ☐ Adding then removing an `Equipment` /song actually changes what prints afterwards
- ☐ `sellSeat()` correctly refuses to double-sell an already-sold seat
- ☐ `getAvailableSeats()` returns the correct sub-collection after some seats are sold
- ☐ `browseConcerts()` prints bookings in date order after adding them out of order
- ☐ Every class has a working, non-default `toString()`
- ☐ Screenshots of the program running in VS Code are captured for the explanation document
- ☐ Explanation document identifies: inheritance, interfaces, overloading, overriding, enums, polymorphism (§7)
- ☐ Explanation references your original week 5 written scenario/use-cases, not just the final version
- ☐ Cover sheet completed with accurate % contribution per student
- ☐ Contribution list included, naming specific classes/sections per student
- ☐ Final code re-checked against the week 1–6 construct list (§9)
- ☐ camelCase for variables/methods, PascalCase for classes, `toString()` on every class — spot-checked